

← VLSIDA/OpenRAM

我们可以直接深入对比其核心策略——特别是 地址分布（Address Distribution）和 Multi-Bank 支持。

🔍 一、核心目标回顾 SRAM 编译器的核心任务之一是：把用户指定的 (AddrWidth, WordWidth) 合理地映射到内部结构——包括 row、col、是否分 core / bank、是否用 column mux 等。这个过程直接决定：WL（字线）...

[Show full text](#)

🔥 Deep

> Thought Process

OpenRAM 地址分布策略与 Multi-Bank 支持深度分析

🔥 一、OpenRAM 的地址分布策略（Python）

1. 入口：sram_config.compute_sizes()

OpenRAM 的地址分布计算入口在 sram_config.py 中的 compute_sizes() 方法：

[sram_config.py:77-106](#)

核心流程：

1. 计算 bitcell 总面积并估算正方形边长
2. 根据边长估算列数，推导 words_per_row
3. 通过 estimate_words_per_row() 获得初始 column mux 配置
4. 通过 amend_words_per_row() 应用行数约束并调整

2. 行/列划分逻辑

estimate_words_per_row() - 启发式估算： [sram_config.py:151-164](#)

- 计算初步的列路数 ($\text{tentative_column_ways} = \text{tentative_num_cols} / \text{word_size}$)
- 从 [1, 2, 4, 8, 16] 中选择最接近的 column mux 大小
- 目标：使阵列接近正方形

amend_words_per_row() - 应用约束： [sram_config.py:166-182](#)

Ask a follow-up question

⚡ Fast ▾

→

- **最大行数**：512 行
- 如果超出范围，动态调整 `words_per_row` 以满足约束

3. 地址组成计算

在 `recompute_sizes()` 方法中计算地址位宽： [sram_config.py:134-141](#)

地址层级结构：

```
Total Address = [Bank Select Bits | Row Address | Column Address]
                  ↳ log2(num_banks) ↳ row_addr_size ↳ col_addr_size ↳
```

- `col_addr_size = log2(words_per_row)` - 列地址（低位）
- `row_addr_size = ceil(log2(num_rows))` - 行地址（中位）
- `bank_addr_size = col_addr_size + row_addr_size` - 单个 bank 内地址
- `addr_size = bank_addr_size + log2(num_banks)` - 总地址宽度（包含 bank 选择位）

4. 是否支持 Multi-Bank?

✅ 完全支持真正的 Multi-Bank

Bank 层级计算： [sram_config.py:82-86](#)

Multi-Bank 包装器实现： [sram_multibank.py:13-34](#)

Verilog 实现的 Bank 选择逻辑： [sram_multibank_template.v:95-120](#)

[sram_multibank_template.v:142-156](#)

关键特性：

- 使用高位地址位 `addr[ADDR_WIDTH-1 : ADDR_WIDTH-BANK_SEL]` 选择 bank
- 每个 bank 有独立的 `csb` 和 `web` 控制信号
- 未选中的 bank 的 `csb` 和 `web` 设为 `1'b1`（禁用状态）
- 使用寄存器化的地址高位进行数据输出多路选择

二、OpenRAM vs YourAM 核心对比

地址公有策略对比

Ask a follow-up question



维度	YourAM (Rust)	OpenRAM (Python)
算法	穷举所有 col_addr_width，选择最优	启发式估算 + 约束调整
Column Mux	$2^{\text{col_addr_width}}$ (0-3) = 1/2/4/8	[1, 2, 4, 8, 16] 中选择最近值
实现方式	直接数学计算 row/col 尺寸	考虑 bitcell 物理尺寸 (height/width)
灵活性	硬编码约束，确定性算法	可通过参数覆盖 words_per_row

2. 约束条件对比

约束类型	YourAM	OpenRAM
行数限制	row <= 64	16 <= row <= 512
列数限制	col <= 128	无显式硬限制（警告超过 16-way mux）
总容量	total_bits <= MAX_BITCELL_SIZE	依赖行列约束隐式限制
约束应用	在 try_one_core 中直接拒绝	在 amend_words_per_row 中动态调整

关键差异：

- ✅ YourAM：严格拒绝超出约束的配置
- 🔄 OpenRAM：尝试调整 words_per_row 以适应约束

3. Multi-Bank/Core 支持对比

特性	YourAM	OpenRAM
术语	Multi-Core	Multi-Bank
独立性	⚠️ 逻辑分片，共享地址/控制	✅ 真正独立的 bank，独立控制信号
最大数量	4 个 Core (2^2)	任意 2^n （理论上）
选择机制	CoreSelector 模块	高位地址位 + case 语句
数据端口	共用数据总线	每个 bank 独立数据输出，顶层 mux
控制信号	全局控制，per-core enable	每个 bank 独立 csb/web gating
使用场景	容量扩展（单一访问）	真正的多 bank 架构（可并行设计扩展）

Ask a follow-up question



维度	YourAM	OpenRAM
实现语言	Rust（类型安全，性能优先）	Python（灵活性，可扩展性）
约束处理	编译期检查，失败即报错	运行时调整，尽量适配
可配置性	受限（硬编码常量）	高（可覆盖 <code>words_per_row</code> 、 <code>num_banks</code> ）
Multi-Bank 语义	容量分片（类似 sub-array）	独立存储模块（标准 multi-bank）

三、总结与建议

OpenRAM 的优势

- 1. **真正的 Multi-Bank 支持**：每个 bank 可独立控制，符合标准 SRAM 架构
- 2. **工艺感知**：基于实际 bitcell 物理尺寸计算，更贴近实际布局
- 3. **灵活的约束处理**：动态调整而非直接拒绝，提高成功率
- 4. **更大的规模支持**：最大 512 行 vs YourAM 的 64 行

YourAM 的优势

- 1. **确定性算法**：穷举搜索保证找到最优 square ratio
- 2. **简洁的实现**：直接数学计算，无需迭代调整
- 3. **编译期安全**：Rust 类型系统保证配置正确性

关键差异

最根本的差异在于 **Multi-Bank 的语义**：

- **YourAM**：逻辑分片，用于突破单个 core 的容量限制
- **OpenRAM**：物理隔离的独立 bank，每个 bank 有独立的控制路径

Notes

OpenRAM 的设计更符合工业界标准的 SRAM 编译器架构，特别是在 multi-bank 支持方面。其地址分布策略考虑了实际物理布局，而不仅仅是数学上的最优。约束条件的动态调整机制也提高了编译器的适用范围。

YourAM 的 "Multi-Core" 更像是 sub-array 或 local bitcell array 的概念，用于内部分片，而非真正的

Ask a follow-up question





Ask a follow-up question



因为youram的模块化做的好，现在希望二次开发，所以youram是否存在不可克服的问题，如果最终目标是一个商用的memory compiler.

⚡ Fast

 [View as codemap](#)

我来分析 YourAM 在商用化道路上可能面临的不可克服问题，并与 OpenRAM 进行对比。

YourAM 面临的商用化挑战

基于之前的对比分析，YourAM 如果要发展成商用 memory compiler，存在以下关键问题：

1. 规模限制问题 ⚠️

硬编码的容量上限：

- 最大行数：64 行（vs OpenRAM 的 512 行）
- 最大列数：128 列
- 最大 Core 数：4 个

这些硬编码限制在商用场景中是**不可接受的**。工业界 SRAM 通常需要支持：

- 数 KB 到数 MB 的容量范围
- 灵活的 bank 配置（8/16/32 banks）
- 可配置的行列比例

OpenRAM 通过参数化设计避免了这个问题，支持 16-512 行的灵活配置，并且可以通过 `words_per_row` 参数覆盖默认行为。 [options.py:52](#)

2. Multi-Bank 架构缺陷 ❌

最致命的问题：YourAM 的 "Multi-Core" 不是真正的 Multi-Bank。

商用 SRAM 编译器必须支持真正的 multi-bank 架构，因为：

- **功耗管理：**未访问的 bank 可以完全关闭

Ask a follow-up question



OpenRAM 的实现符合工业标准: [sram_multibank_template.v:151-155](#)

每个 bank 有独立的片选和写使能控制, 未选中的 bank 被完全禁用 (`csb = 1'b1, web = 1'b1`)。
[sram_multibank_template.v:141-155](#)

3. 工艺移植性问题 🛠️

YourAM 的数学优化方法 (最小化 $|row - col|$) 忽略了实际工艺参数:

- Bitcell 的物理尺寸 (height vs width)
- 金属层的电阻/电容特性
- 工艺角的时序变化

OpenRAM 基于实际 bitcell 物理尺寸计算, 更贴近真实布局。这在商用编译器中至关重要, 因为不同工艺节点的 bitcell 长宽比差异很大。

4. 缺少完整的 EDA 集成 📦

商用 memory compiler 必须提供:

- Liberty (.lib) 文件用于综合
- LEF 文件用于布局布线
- SPICE 表征支持多个 PVT corner [options.py:40-45](#)
- DRC/LVS 验证流程 [index.md:74-79](#)

DeepWiki

Share



- 负载/转换时间扫描 (load_scales, slew_scales) [options.py:44-45](#)
- 自动化 DRC/LVS 检查
- HTML datasheet 生成

5. 可扩展性架构问题 🏗️

OpenRAM 的模块化设计允许:

- 自定义 bitcell 替换 [library_cells.md:33-42](#)
- 技术文件独立管理 [index.md:84-92](#)

Ask a follow-up question



商用化路径建议

短期可解决的问题

1. 移除硬编码限制：将 `MAX_ROW_SIZE`、`MAX_COLUMN_SIZE` 改为可配置参数
2. 增加 column mux 选项：支持 16-way、32-way mux
3. 添加工艺参数：引入 bitcell 物理尺寸作为输入

需要架构重构的问题

1. 重新设计 Multi-Bank：实现真正的独立 bank 架构
2. 添加表征系统：集成 SPICE 仿真和 Liberty 生成
3. 工艺移植框架：建立类似 OpenRAM 的技术目录结构

不可克服的根本问题

设计哲学冲突：

- YourAM 追求数学最优 (square ratio)
- 商用编译器需要工艺感知的实用最优

这需要从根本上改变优化目标，可能需要重写核心算法。

结论

YourAM 的模块化设计确实很好，但要成为商用 memory compiler，需要解决：

1. 致命问题：Multi-Bank 架构必须重新设计
2. 严重问题：容量限制、工艺移植性
3. 必要功能：完整的 EDA 集成 (Liberty/LEF/DRC/LVS)

OpenRAM 经过多年发展，已经在学术界和工业界得到验证，有多篇论文支持。 [README.md:59-60](#)
其架构设计更符合商用需求。

建议：如果目标是商用，可以考虑：

1. 参考 OpenRAM 的架构重构 YourAM
2. 或者将 YourAM 定位为教学/研究工具，而非商用编译器

Ask a follow-up question



OpenRAM 是 BSD 3-Clause 开源许可证，允许商业使用。 [README.md:53](#) 其文档和社区支持也更完善，有活跃的用户组和开发者组。 [README.md:45-47](#) 这些都是商用化的重要考量因素。

YourAM 的 Rust 实现在性能和类型安全方面有优势，但 Python 的灵活性在 EDA 工具开发中更受欢迎，因为需要频繁与各种外部工具（SPICE 仿真器、DRC/LVS 工具）集成。

Wiki pages you might want to explore:

- [Usage and Basic Workflow \(VLSIDA/OpenRAM\)](#)
- [Delay and Power Characterization \(VLSIDA/OpenRAM\)](#)



✓ VLSIDA/OpenRAM compiler/sram_config.py

⌵

```
74         for member in members:
75             setattr(module, member, getattr(self, member))
76
77     def compute_sizes(self):
78         """ Computes the organization of the memory using bitcell size by trying to make it s
79
80         bitcell = factory.create(module_type=OPTS.bitcell)
81
82         debug.check(ceil(log(self.num_banks, 2)) == log(self.num_banks, 2) ,
83                     "Number of banks should be power of 2.")
84
```

Ask a follow-up question



```

92         self.bank_side_length = sqrt(self.bank_area)
93
94         # Estimate the words per row given the height of the bitcell and the square side l
95         self.tentative_num_cols = int(self.bank_side_length / bitcell.width)
96         self.words_per_row = self.estimate_words_per_row(self.tentative_num_cols, self.wor
97
98         # Estimate the number of rows given the tentative words per row
99         self.tentative_num_rows = self.num_bits_per_bank / (self.words_per_row * self.worc
100         self.words_per_row = self.amend_words_per_row(self.tentative_num_rows, self.words
101
102         self.recompute_sizes()
103
104         # Set word_per_row in OPTS
105         OPTS.words_per_row = self.words_per_row
106         debug.info(1, "Set SRAM Words Per Row={}".format(OPTS.words_per_row))
107
108     def recompute_sizes(self):
109         """

```

↓

↑

```

131         self.write_size = self.word_size
132
133         # Compute the address and bank sizes
134         self.row_addr_size = ceil(log(self.num_rows, 2))
135         self.col_addr_size = int(log(self.words_per_row, 2))

```

✖  VLSIDA/OpenRAM compiler/options.py 

↑

```

25     # This is the name of the technology.
26     tech_name = ""
27
28     # Port configuration (1-2 ports allowed)
29     num_rw_ports = 1
30     num_r_ports = 0
31     num_w_ports = 0
32
33     # By default, don't use hierarchical wordline
34     local_array_size = 0
35
36     # Write mask size, default will be overwritten with word_size if not user specified
37     write_size = None
38
39     # These will get initialized by the user or the tech file
40     nominal_corner_only = False
41     supply_voltages = ""
42     temperatures = ""
43     process_corners = ""
44     load_scales = ""
45     slew_scales = ""
46
47     # Size parameters must be specified by user in config file.
48     # num_words = 0
49     # num_bits_per_bank = 0

```

Ask a follow-up question

▽